CS 450/535: Assignment 03

Programming Assignments (85%)

Initial Setup

- **This assignment is in C++.**
- Copy src/app/Assign02.cpp and name it **src/app/Assign03.cpp**
- Change the application name to "**Assign03**"
- Change the window title to be "Assign03: " + your SITNET ID
    - E.g., "Assign03: realemj"
- Make a copy of the vulkanshaders/Assign02 folder and name it **vulkanshaders/Assign03**
- Modify **CMakeLists.txt** by adding the following line to the end of the file:
    - CREATE_VULKAN_EXECUTABLE(Assign03)
- Make sure the program configures, compiles, and runs as-is
- **Remove ALL additional adjustments to the HOST vertices from the previous assignment**

Requirements

Both CS 450 and 535

Assign03.cpp

- Create a struct to hold vertex shader push constants: **UPushVertex**
    - Add one field for the model matrix:
      alignas(16) glm::mat4 modelMat
    - Add one field to determine how much of color is from normals:
      alignas(16) float useNormalAsColor;

- Add the following globals:
    - Extra Z rotation: **float rotAngleZ = 0.0f;**
    - Extra Z translation: **float zOffset = 0.5f;**
    - Weight for using normals as the color: **float useNormalAsColor = 1.0f;**
- Change **useFrontCCW** to **false** by default

- Add the following keys to the **key_callback()** function:
    - If the action is either GLFW_PRESS or GLFW_REPEAT, check for the following keys:
        - GLFW_KEY_J
            - Add 1.0 to rotAngleZ
        - GLFW_KEY_K
            - Subtract 1.0 from rotAngleZ

        - GLFW_KEY_O
            - Add 0.1 to useNormalAsColor BUT clamp to [0,1]
        - GLFW_KEY_I (letter i)
            - Subtract 0.1 from useNormalAsColor BUT clamp to [0,1]
        - GLFW_KEY_U
            - Add 0.05 to zOffset
        - GLFW_KEY_Y
            - Subtract 0.05 from zOffset

- Add a new function: **glm::mat4 assimpToGLM(aiMatrix4x4 &a)**
    - Convert from row-major aiMatrix4x4 to column-major glm::mat4

- Add the following function for generating a transformation to rotate around the LOCAL Z axis: **glm::mat4 makeRotateZ(float degrees, glm::vec3 offset)**
    - Generate transformation matrices (with glm) and form a composite transformation to perform the following IN THE CORRECT ORDER:
        - Translate by NEGATIVE offset
        - Rotate degrees around the Z axis
            - **REMEMBER TO CONVERT degrees to RADIANS!!!!**
        - Translate by offset
    - Return the composite transformation

- Create a new function:
  void renderSceneNode(const vk::raii::CommandBuffer &commandBuffer, pro::VulkanPipelineData &pipelineData, vector<pro::VulkanMesh> &allMeshes, aiNode *node, glm::mat4 parentMat, int level)
    - ***CS 535 only: You may modify the above prototype to accommodate passing in the topology. You may also modify it to handle the individual node rotations (discussed later).***
    - Get the transformation for the current node, which is an aiMatrix4x4: ***node->mTransformation***
    - Convert the transformation to a ***glm::mat4 nodeT*** using the **assimpToGLM**() function
    - Compute the current model matrix: ***glm::mat4 modelMat = parentMat*nodeT***
    - Get location of current node by:
        - Grabbing the last column of modelMat, which is a vec4
            - Remember that glm matrices are stored in column-major format, so modelMat[3] gives you the last column.
        - Convert this vec4 to a vec3 pos
    - Call **makeRotateZ(rotAngleZ, pos)** to get a proper local Z rotation: ***R***
    - ***CS 535 only: apply node-specific Z rotation (discussed later).***

- Add in the Z translation based on zOffset: **R = glm::translate(glm::vec3(0,0,zOffset))*R;**
- Generate a temporary model matrix model matrix as:
  - ***glm::mat4 tmpModel = R * modelMat***
- Create an instance of **UPushVertex** and store ***tmpModel*** as the model matrix as well as the current value of useNormalAsColor.
- Use **commandBuffer.pushConstants()** to push up the UPushVertex data
- For each mesh in the NODE (***node->mNumMeshes*** meshes total)
  - Get the index of the mesh: ***int index = node->mMeshes[i]***
  - ***CS 535 only: Set the correct topology here.***
  - Call **pro::recordDrawVulkanMesh()** on each mesh ***allMeshes[index]***
- Call renderSceneNode() on each child of the NODE (***node->mNumChildren*** children total)
  - Command buffer, pipeline data, all meshes: same as passed in
  - Node: ***node->mChildren[i]***
  - Parent matrix: ***modelMat*** (NOT tmpModel!)
  - Level: ***level + 1***

- Change **recordFrame():**
  - Add **const aiScene *scene** as a parameter.
  - Instead of looping through and drawing all meshes, call renderSceneNode() ONCE:
    - Node: ***scene->mRootNode***
    - Parent matrix: ***glm::mat4(1.0)***
    - Level: ***0***

- **In the main function:**
  - **IN PIPELINE SETUP SECTION:**
    - Set the correct push constant ranges.
  - **IN THE RENDER LOOP:**
    - When calling **recordFrame(),** remember to also pass in the loaded scene object.

shader.vert

- Add the appropriate push constant struct.
- For **gl_Position**, multiply **pc.modelMat** by the input vertex position.
- Compute the worldNormal with the **correct normal transform** version of pc.modelMat
  - You can calculate it locally: use the transpose of the inverse of the upper-3x3 matrix.
- Pass the worldNormal along as the interNormal.
- Get the colorNormal by rescaling worldNormal from [-1,1] to [0,1] and making it a vec4 with alpha 1.0f.
- Change interColor to be a mix of color and colorNormal based on pc.useNormalAsColor: **interColor = mix(color, colorNormal, pc.useNormalAsColor);**

shader.frag

- Make sure out_color is set to interColor (instead of the previous assignment's normal-only setup).

CS 535 Only

In addition to the above, you will also implement code that will allow the user to **change the Z rotation of INDIVIDUAL node transforms**.

- Add two new keyboard commands:
    - GLFW_KEY_PERIOD
        - Increment current node index
    - GLFW_KEY_COMMA
        - Decrement current node index

Any change to the current node index should print the index to the console.

Make sure the index does not go out of bounds, EXCEPT that -1 means rotations are applied to ALL nodes (NOT the root node, but each and every node individually).

Rotations should be persistent. In other words, if you change the rotation of node 2, and then change to node 3, node 2's rotation SHOULD remain the same regardless of how you change node 3.

The one obvious exception is when node index is -1, but even then, rotations should be ADDED onto existing individual rotations!

You can implement this largely as you see fit, provided it is a reasonable and well thought out design. Also, you may decide how the node index is applied to the tree structure, PROVIDED that each node index refers to only ONE node in the tree.

Only Z rotations need to be node specific. The Z offset is applied the same to all nodes. **However, make sure that all rotations are done BEFORE applying the Z offset translation.**

Hint: you may make reasonable changes to renderSceneNode (e.g., have it return a value rather than void) if that helps.

Some nodes do not have meshes, and therefore you will not see any rotation effects. This is normal. (Remember that we DON'T pass along the extra rotations to the child nodes.)

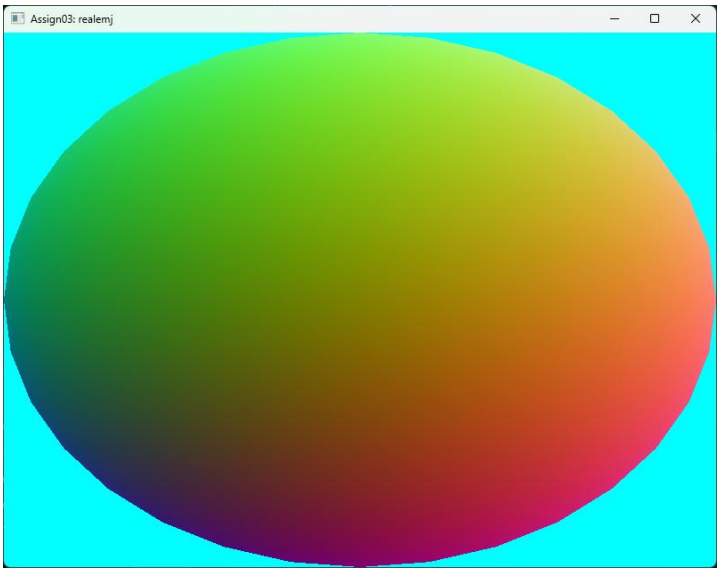
Screenshots (10%)

- **Take screenshots** of the application window and copy them to the **screenshots** folder
    - *Windows:* Alt + Screenshot button OR Windows Button + Shift + S

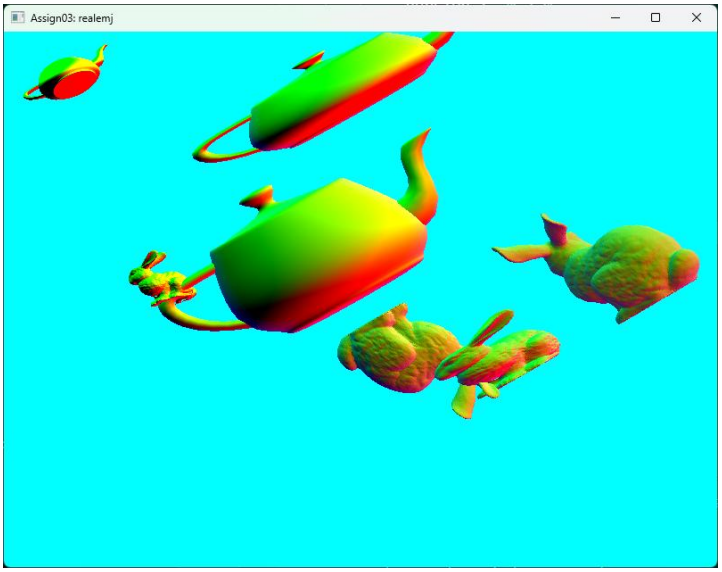
CS 450 Only

You should have **TWO** screenshots total:

- sphere.obj after moving sphere back so it is visible → **Assign03_sphere.png**



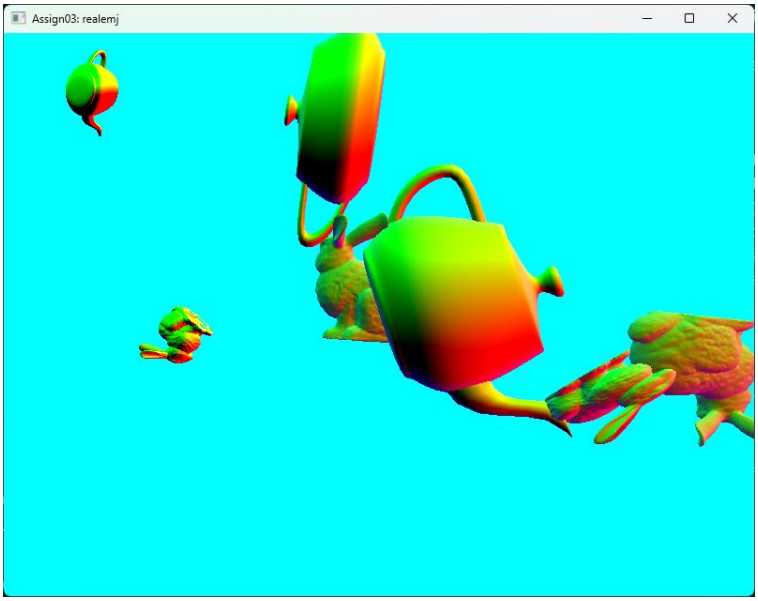
- bunnyteatime.glb after J rotation → **Assign03_bunnyteatime_J.png**



CS 535 Only

You should have **THREE** screenshots total: the same two as CS 450 PLUS:

- bunnyteatime.glb after doing J rotations on DIFFERENT items → **Assign03_bunnyteatime_mix.png**



README (5%)

- Under the "Applications" section, add a subsection very briefly describing the assignment.
    - ***Also include your screenshots in the document as an <img> (as we have done with the exercises)***

Grading

Your OVERALL assignment grade is weighted as follows:

- 85% - Programming
- 10% - Screenshots
- 5% - README

I reserve the right to take points off for not meeting the specifications in this assignment description.

The following specific penalties (applied to the WHOLE assignment) can be expected:

Problem	Maximum Penalty
Code that does not compile	60
One GIGANTIC commit for whole assignment	30
Incorrect renderSceneNode()	25

Runs with Vulkan validation warnings/errors	20
Keyboard functions incorrect	20
CS 535: Incorrect individual rotations	15
Incorrect vertex shader	15
Incorrect makeRotateZ	10
Missing screenshots	10
Incorrect zOffset	5
Incorrect fragment shader	5
README update missing/incorrect	5
Missing assignment Git branch	5
Failed to merge assignment branch	2.5
Incorrect application name	2
Incorrect window title	2
Wrong names for branch, files, etc.	2 for each
Bad commit messages	2 for each

In general, these are things that will be penalized:

- **Code that does not compile (up to 60 points off!)**
- Sloppy or poor coding style
- Bad coding design principles
- Code that crashes, does not run, or takes a VERY long time to complete
- Using code from ANY source other than the course materials
- Collaboration on code of ANY kind; this is an INDIVIDUAL PROJECT
- Sharing code with other people in this class or using code from this or any other related class
- Output that is incorrect
- Algorithms/implementations that are incorrect
- Submitting improper files
- Failing to submit ALL required files